



TP – Exception

Exercice 1

Soit la classe Temps suivante :

```
class Temps {  
  
    private int heures;  
    private int minutes;  
    private int secondes;  
  
    Temps(int h, int m, int s) {  
        heures = h;  
        minutes = m;  
        secondes = s;  
    }  
  
    public static void main(String[] args) {  
        Temps t = new Temps(4, 12, 67);  
    }  
}
```

1. Modifier le constructeur de cette classe de manière qu'il lance une exception de type **TempsException** si les heures, les minutes ou les secondes ne correspondent pas à un temps valide. Cette exception ne sera pas traitée localement.
2. Modifier le code de la méthode main de manière que l'exception TempsException soit traitée en affichant le message suivant : "Temps invalide".

Exercice 2

1. Considérons la classe **CalcFactorielle** définie ci-dessous. Ecrire la méthode `main()` qui doit récupérer un entier `n` sur la ligne de commande, appelle la méthode `fact(n)` et ensuite affiche `n!`.

Programme Java	Exemples d'exécution :
<pre>public class CalcFactorielle { public static int fact(int k) { // calcule k! int f=1; // f=k! for (int i=1; i<=k; i++) f=f*i; return f; } public static void main(String[] args) { // ... } }</pre>	<pre>> java CalcFactorielle 6 6! = 720 > java CalcFactorielle -2 -2! = 1 > java CalcFactorielle 5.8 Exception in thread "main" java.lang.NumberFormatException: 5.8 > java CalcFactorielle Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException</pre>

2. Dans cette question on s'intéresse à la gestion des différentes erreurs qui peuvent se produire.
 - Modifier la méthode `fact()` pour qu'elle puisse générer une exception de type «ExceptionNegatif» (classe à définir) si on veut calculer la factorielle d'un entier négatif.
 - Réécrire la méthode `main()` afin de traiter toutes les exceptions mentionnées sur les exemples d'exécution ci-dessus.

Exercice 3

Ecrire une classe **Entreprise**. Une entreprise a un nombre d'employés, un capital, un nom, une mission, et une méthode **public String mission()** qui déclare le lancement de cette exception **SecretMissionException**. On aura également une Méthode **public int capital()** qui renvoie le capital et qui déclare le lancement de l'exception **NonProfitException**.

Ecrire une classe **EntrepriseSecrete** qui hérite d'**Entreprise** et dont la méthode `mission` lance l'exception **SecretMissionException**. Ecrire une classe **EntrepriseSansProfit** qui hérite d'**Entreprise** et dont la méthode `capital` lance l'exception **NonProfitException**.

Ecrire une classe **Entreprises** ayant une méthode **tousLesEntreprises(Entreprise[] e)** qui prend en entrée un tableau

d'entreprises et affiche la mission et le capital de toutes les entreprises, si c'est possible. Tester cette dernière méthode sur les entreprises suivantes : "Ooredoo" "Poulina" "Meublatex" et " Monoprix". Ajouter également à ce test l'entreprise secrète "CIA" et l'entreprise sans profit "CroixRouge".